in vscode how do I set up the visual were I see a vertical line that demarks when my line of code is too long

In VS Code, that vertical line is called a **ruler**. You can configure it to appear at whatever column length you want (e.g., 80, 100, 120 characters).

Quick setup (UI)

1. Open **Settings**
   - Press **Ctrl** + **,** (Windows/Linux) or **Cmd** + **,** (Mac)
2. In the search bar, type **rulers**
3. Find **Editor: Rulers**
4. Click **Add Item** and enter a number (e.g., `80` or `100`)

You'll immediately see a vertical guideline at that column.

Set it via settings.json (recommended for precision)

1. Press **Ctrl** + **Shift** + **P**
2. Type **Preferences: Open Settings (JSON)**
3. Add this:

```json
"editor.rulers": [80]
```

Copy code

You can add multiple rulers:

```json
"editor.rulers": [80, 100, 120]
```

Copy code

Language-specific rulers (optional)

If you only want it for a specific language (e.g., Python):

```json
"[python]": {
    "editor.rulers": [79]
}
```

Copy code

Bonus (auto-format line length)

If you also want formatting tools to respect line length:

- For Python: set `"python.formatting.blackArgs": ["--line-length", "88"]`
- For Prettier (JS/TS):

```json
"prettier.printWidth": 100
```

Copy code

If you tell me what language you're working in, I can suggest the typical line length and a clean VS Code setup for it.

I want to make an N-body problem simulator in C++, the graphics are going to be quite simple becuase it will be 2D and I am just going to make some simple little white balls orbit each other. It will have the same aesthetics as pong. What is a good graphics library and linear algebra library in C++ that I can use for this project

For your project (2D N-body with **Pong-style visuals**), you want:

- **Minimal boilerplate**
- Easy window + drawing circles

- Good performance
- Clean C++ API

Here's the **best combo** for your use case, plus alternatives and why.

Recommended Stack

Graphics: SFML

Why it's ideal

- Designed for simple 2D graphics and games
- Clean, object-oriented C++ API
- Cross-platform (Windows, macOS, Linux)
- Hardware-accelerated 2D rendering
- Handles windowing, input, timing, and drawing shapes out of the box Wikipedia

You can literally draw bodies like this:

```cpp
sf::CircleShape body(radius);
body.setFillColor(sf::Color::White);
body.setPosition(x, y);
window.draw(body);
```

Copy code

SFML is specifically intended to provide a **simple API for multimedia and 2D graphics**, internally using OpenGL so you don't deal with low-level graphics yourself. Wikipedia

Why it fits your project

- Pong-like aesthetics = shapes, no textures needed
- Fast iteration
- Minimal setup compared to OpenGL

Linear Algebra: Eigen

This is the **standard choice** for scientific C++.

- Header-only (no build hassle)

- Fast matrix/vector operations
- Supports geometry transforms, solvers, etc.
- Uses compile-time optimizations and vectorization Wikipedia

Perfect for:

- Position/velocity vectors
- Force calculations
- Integrators (Euler, RK4, Verlet)

Example:

```cpp
#include <Eigen/Dense>

Eigen::Vector2d pos, vel, acc;
pos += vel * dt;
vel += acc * dt;
```

Copy code

Why this combo is especially good for an N-body sim

Need	Library
Draw circles, window loop	SFML
Timing (dt)	SFML
Vector math	Eigen
Physics integration	Eigen

This gives you a clean separation:

- **Eigen = physics**
- **SFML = visualization**

Good Alternatives (if you're curious)

Graphics

- **SDL2** – more low-level, more setup
- **raylib** – very simple (C library, works with C++)
- **OpenGL + GLFW** – overkill for this project

Community consensus often recommends **SFML, SDL, or raylib** for simple 2D work. Reddit

Linear algebra alternatives

- **GLM** – OpenGL-style math library
- **Armadillo** – MATLAB-like syntax
- **Hand-rolled Vec2** – honestly fine for small N-body sims

But Eigen is the safest long-term choice.

What I'd personally use for your exact project

SFML + Eigen

This combo is:

- Clean
- Minimal friction
- Common in teaching and scientific projects
- Perfect for a portfolio simulation

Bonus architecture suggestion (for N-body)

text

```
main.cpp
  ├── Simulation (Eigen math)
  ├── Renderer (SFML)
  └── Time loop
```

Copy code

Update loop:

cpp

```
while (window.isOpen()) {
    handleEvents();
    simulation.step(dt);   // physics (Eigen)
    renderer.draw();       // SFML
}
```

Copy code

If you want, I can give you a **minimal starter template (SFML + Eigen)** that shows:

- Window
- One orbiting body
- Fixed timestep

  It's about ~80 lines and a great starting point.

Sources

can you use SFML on windows subsystem for linux and get a window to open up natively on my windows machine

Yes — but **how well this works depends on which WSL version you're using**.

Short answer:

- **WSL2 + WSLg (Windows 11)** → Yes, works out of the box.
- **Windows 10 or older WSL without WSLg** → Possible, but requires an external X server and extra setup.

Best Case: Windows 11 with WSLg (Recommended)

If you're on **Windows 11**, Microsoft provides **WSLg**, which allows Linux GUI apps to open **natively on your Windows desktop**.

SFML windows will:

- Open like normal Windows apps

- Support hardware acceleration (OpenGL)
- Require almost no configuration

Step 1 — Install SFML in WSL

In your WSL terminal:

bash

```
sudo apt update
sudo apt install libsfml-dev
```

Copy code

Step 2 — Test with a minimal program

cpp

```
#include <SFML/Graphics.hpp>

int main() {
    sf::RenderWindow window(sf::VideoMode(800, 600), "SFML WSL Test");

    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        window.clear(sf::Color::Black);
        window.display();
    }
}
```

Copy code

Compile:

bash

```
g++ main.cpp -o test -lsfml-graphics -lsfml-window -lsfml-system
```

Copy code

Run:

bash

```
./test
```

Copy code

If WSLg is active, a window will appear **on your Windows desktop**.

How to Check if WSLg is Available

Run:

bash

```
echo $WAYLAND_DISPLAY
```

Copy code

If [illegible]


```
wayland-0
```

Copy code

You're using WSLg.

You can also run:

bash

```
glxinfo | grep OpenGL
```

Copy code

(Install with `sudo apt install mesa-utils` if needed.)

If You're on Windows 10 (No WSLg)

You'll need an **X server**:

Options:

- VcXsrv
- X410 (paid)
- GWSL

Basic workflow:

1. Install VcXsrv on Windows
2. Start it
3. In WSL:

bash Copy code

```bash
export DISPLAY=$(grep nameserver /etc/resolv.conf | awk '{print $2}'):0
```

Then run your SFML app.

But:
Performance and OpenGL support can be inconsistent.
For graphics projects, this setup is often frustrating.

Recommendation for Your N-Body Project

If your goal is:

- Real-time rendering
- Smooth animation
- OpenGL acceleration

Best options:

Option	Recommendation
Windows 11 + WSLg	Good
Windows 10 + X server	Not ideal
Native Windows build (Visual Studio / MinGW)	**Best performance**

For graphics development, compiling **natively on Windows** is usually smoother than WSL.

Practical Advice

If you're doing serious graphics work:

- Develop physics in WSL if you like Linux
- But consider building the SFML app **natively on Windows**

If you want, I can also give you:

- A **minimal CMake setup** for SFML + Eigen (cross-platform), or
- A **project structure for an N-body simulator**, or
- A quick comparison: **SFML vs raylib for this exact project**.